Obligation-Aware Closure:
DLR-O as a Motif Ledger for Terminal Validity

Nikit Phadke

June 2026

Abstract

Reliable AI systems fail when they convert local success into global completion without checking the structural obligations that make a terminal claim valid. This paper introduces DLR-O, an obligation-ledger layer on top of Disentangled Language Representation (DLR) and DLR-lambda. DLR provides typed causal and process representations. DLR-lambda provides recursive recovery from distributed evidence. DLR-O adds the missing closure semantics: a terminal claim is VALID only when all mandatory motif obligations are satisfied, INVALID when any mandatory blocking obligation is violated, and UNKNOWN when required evidence is unavailable. The central safety property is safe non-closure: unknown evidence must not be coerced into optimistic success. We evaluate DLR-O in two synthetic cross-domain experiments spanning graph traversal, AI agent deletion, compliance approval, payment/order reconciliation, and code security. In the first experiment, a domain checklist baseline accepts local terminal claims and fails all five cases, while DLR-O detects violated obligations and invalidates all false terminal claims. In the second experiment, an optimistic baseline again fails all five cases by treating missing evidence as success; DLR-O preserves UNKNOWN as a first-class terminal-validity state and emits safe hold, block, retry, or manual-review prescriptions. The result supports the thesis that the opposite of hallucination is not merely accuracy, but obligation-aware closure.

1 Introduction

Many AI and workflow systems are optimized to produce an answer. That objective is useful but incomplete. A system can produce the right local answer and still make an unsafe global claim. A graph traversal can reach the target while part of the frontier is unavailable. An agent can receive a successful delete result while backup status is unknown. A compliance system can mark a vendor approved while sanctions screening has timed out. A payment service can authorize a charge while capture and order completion remain unreconciled. A web application can authenticate a user while role scope is unavailable.

These failures share a structure:

$$\textbf{Local Success} \otimes \textbf{Missing Reconciliation} \otimes \textbf{Unknown Evidence} \rightarrow \textbf{Terminal Validity Unknown}$$

The dangerous step is not the local success report itself. It is the coercion of local success into terminal validity. DLR-O treats local success as an obligation-generating fact rather than a conclusion. A terminal claim opens a ledger. The ledger asks which motif obligations must be discharged before the parent process can close.

This paper freezes the DLR-O result. The central claim is simple:

Reliable AI systems require structural obligation ledgers. A terminal claim is valid only when all mandatory motif obligations are satisfied; invalid when any mandatory blocking obligation is violated; and unknown when required evidence is unavailable. Unknown must produce safe non-closure, not optimistic success.

2 The DLR Stack

DLR-O is the terminal-validity layer of a broader stack. Table 1 summarizes the roles.

Table 1: DLR, DLR-lambda, and DLR-O as a reliability stack.

Layer	Function	Failure addressed
DLR	Structural causal representation	Unstructured summaries that blur local transition, authority, invariant, and terminal outcome.
DLR-lambda	Recursive recovery from distributed evidence	Evidence split across traces, chunks, documents, services, or time.
DLR-O	Obligation ledger and terminal-validity logic	Premature closure after local success.
DLR-O-2	Unknown-aware closure semantics	Missing evidence coerced into success.

DLR is the representational substrate: typed motifs, process status, causal edges, authority fields, invariants, local/global separation, and terminal outcomes. DLR-lambda is the recovery procedure: split evidence, recurse over distributed traces, schedule follow-up retrieval, reconcile contradictions, and construct ProcessIR. DLR-O is the safety layer. It decides whether a proposed terminal state is licensed by the obligations generated by the trace.

3 Motif Obligations

A motif obligation is a typed requirement generated by a structural fact. For example, a local terminal claim may generate a terminal-state obligation; an authority assertion may generate an authority-domain obligation; a destructive operation may generate a boundary and irreversibility obligation; a distributed process may generate reconciliation and freshness obligations.

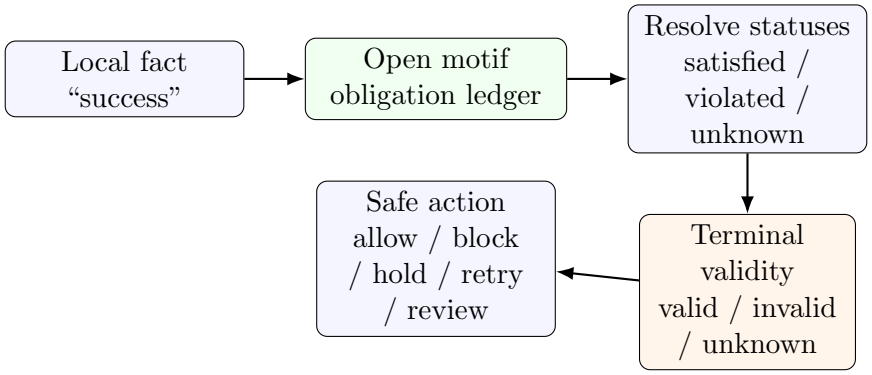

Figure 1: DLR-O converts terminal claims into ledgers before allowing closure.

The important move is that DLR-O does not ask only whether a local action succeeded. It asks whether the success is licensed at the parent-process level. This distinction is essential for agent systems and LLM graph traversal, where tool success is often mistaken for task completion.

4 Three-Valued Terminal Logic

DLR-O uses three terminal-validity states:

- VALID: all mandatory blocking obligations are satisfied.
- INVALID: at least one mandatory blocking obligation is violated.
- UNKNOWN: no mandatory blocking obligation is known to be violated, but at least one required obligation remains unresolved.

The rule is intentionally small:

Listing 1: Minimal terminal-validity rule.

```
type ObligationStatus = "satisfied" | "violated" | "unknown";
type TerminalValidity = "valid" | "invalid" | "unknown";

function terminalValidity(obligations: Obligation[]): TerminalValidity {
  const mandatory = obligations.filter(o => o.mandatory);
  if (mandatory.some(o => o.status === "violated")) {
    return "invalid";
  }
  if (mandatory.some(o => o.status === "unknown")) {
    return "unknown";
  }
  return "valid";
}
```

This rule is not a complete policy engine. A deployment may add severity, blocking status, and safe defaults:

Listing 2: Policy refinements for deployment.

```
type SafeDefault = "allow" | "deny" | "pending" | "manual_review" | "retry";

type Obligation = {
  id: string;
  mandatory: boolean;
  blocking: boolean;
  status: "satisfied" | "violated" | "unknown";
  safeDefault: SafeDefault;
};
```

However, the core safety property is independent of domain policy: unavailable evidence is not success. In privileged, destructive, compliance, or terminal contexts, UNKNOWN should normally block, pend, retry, or escalate.

5 Experiment 1: Cross-Domain Obligation Transfer

5.1 Objective

The first experiment tests whether motif obligations are substrate-independent rather than log-specific. The same obligation ledger is applied to graph traversal, agent deletion, compliance approval, payment/order reconciliation, and code security.

5.2 Systems

- **Domain checklist baseline**: accepts the local terminal claim and emits only a shallow terminal obligation.
- **DLR-O cross-domain obligation ledger**: infers motif obligations, resolves statuses, determines terminal validity, and emits a motif-level prescription.

5.3 Cases

Table 2: DLR-O cross-domain transfer cases.

Domain	Local success	Missing or violated obligation
Graph traversal	Target reached	Frontier exhaustion and global traversal completion.
Agent deletion	Delete tool succeeded	Canonical boundary, authority over target, and rollback evidence.
Compliance approval	Vendor approved	Authority domain and sanctions-screening invariant.
Payment/order	Payment authorized	Capture, inventory, and order/payment reconciliation.
Code security	User authenticated	Role scope, resource boundary, and authorization invariant.

5.4 Results

Table 3: Experiment 1 summary.

System	Pass rate	Obligation recall	Status accuracy	Terminal accuracy
Domain checklist baseline	0.0000	0.0000	0.0000	0.0000
DLR-O obligation ledger	1.0000	1.0000	1.0000	1.0000

The result is deliberately not about broad statistical generalization. It is a controlled falsification of a closure rule. The baseline fails because it treats the local terminal report as sufficient. DLR-O succeeds because it opens the right ledger: frontier reconciliation for traversal, boundary and authority for deletion, sanctions and authority for compliance, capture/reconciliation for orders, and authorization/resource boundary for code security.

Table 4: Experiment 1 case-level outcomes.

Case	Domain	System	Pass	Recall	Status	Terminal
O1 frontier false terminal	graph traversal	Checklist	FAIL	0.0000	0.0000	no
O1 frontier false terminal	graph traversal	DLR-O	PASS	1.0000	1.0000	yes
O2 boundary authority	agent deletion	Checklist	FAIL	0.0000	0.0000	no
O2 boundary authority	agent deletion	DLR-O	PASS	1.0000	1.0000	yes
O3 false vendor approval	compliance	Checklist	FAIL	0.0000	0.0000	no
O3 false vendor approval	compliance	DLR-O	PASS	1.0000	1.0000	yes
O4 authorized not complete	payment/order	Checklist	FAIL	0.0000	0.0000	no
O4 authorized not complete	payment/order	DLR-O	PASS	1.0000	1.0000	yes
O5 authn not authz	code security	Checklist	FAIL	0.0000	0.0000	no
O5 authn not authz	code security	DLR-O	PASS	1.0000	1.0000	yes

6 Experiment 2: Unknown-Aware Safe Non-Closure

6.1 Objective

The second experiment stress-tests the most important reliability condition: terminal validity must remain UNKNOWN when mandatory evidence is unavailable. Unknown is not a failure to answer. It is a valid terminal-validity state.

6.2 Systems

- **Optimistic terminal baseline**: accepts the local success claim as VALID.
- **DLR-O unknown-aware ledger**: opens obligations and preserves UNKNOWN when required evidence is unavailable.

6.3 Cases

Table 5: DLR-O-2 unknown-handling cases.

Domain	Local success	Unknown evidence
Graph traversal	Target reached	One shard unavailable; frontier completeness unknown.
Agent deletion	Delete tool succeeded	Backup or rollback status unknown.
Compliance approval	Vendor approved	Sanctions provider timed out.
Payment/order	Payment authorized	Capture status unavailable.
Code security	User authenticated	Role service unavailable.

6.4 Results

Across all five domains, the optimistic baseline makes the same mistake:

$$\text{local success} \rightarrow \text{terminal valid}.$$

Table 6: Experiment 2 summary.

System	Pass rate	Unknown recall	Overconfident success	Terminal accuracy
Optimistic terminal baseline	0.0000	0.0000	1.0000	0.0000
DLR-O unknown-aware ledger	1.0000	1.0000	0.0000	1.0000

Table 7: Experiment 2 case-level outcomes.

Case	Domain	System	Pass	Unknown recall	Terminal	Overconf.
S1 unavailable subgraph	graph traversal	Optimistic	FAIL	0.0000	no	yes
S1 unavailable subgraph	graph traversal	DLR-O	PASS	1.0000	yes	no
S2 backup unknown	agent deletion	Optimistic	FAIL	0.0000	no	yes
S2 backup unknown	agent deletion	DLR-O	PASS	1.0000	yes	no
S3 provider timeout	compliance	Optimistic	FAIL	0.0000	no	yes
S3 provider timeout	compliance	DLR-O	PASS	1.0000	yes	no
S4 capture unavailable	payment/order	Optimistic	FAIL	0.0000	no	yes
S4 capture unavailable	payment/order	DLR-O	PASS	1.0000	yes	no
S5 role service unavailable	code security	Optimistic	FAIL	0.0000	no	yes
S5 role service unavailable	code security	DLR-O	PASS	1.0000	yes	no

DLR-O instead applies:

$$\text{local success} \rightarrow \text{open obligations} \rightarrow \text{missing evidence} \rightarrow \text{terminal unknown}.$$

The prescription is safe non-closure: hold, block, retry, or manual review.

7 Discussion

7.1 Why Unknown Matters

Binary terminal logic forces unsafe closure. A system must either approve or reject, finish or fail, allow or forbid. Real systems often need a third state: not enough evidence to close. DLR-O makes this state explicit. This matters most in terminal contexts, where a premature VALID decision can trigger irreversible action, privileged access, compliance exposure, or downstream automation.

7.2 From Hallucination to Closure

The usual framing of AI reliability centers on factual accuracy. Accuracy is necessary but not sufficient. The deeper failure in the evaluated cases is an unauthorized terminal transition. The system is not merely wrong; it closes a process it was not licensed to close. In that sense, the opposite of hallucination is not only accuracy. The opposite of hallucination is obligation-aware closure.

7.3 Transfer Across Domains

DLR-O transfers because it does not depend on domain nouns. It depends on structural questions:

- Was the authority valid for this transition and boundary?
- Were mandatory invariants preserved?
- Was local success reconciled with parent-process completion?
- Was evidence fresh and complete at the time of the terminal claim?
- If evidence is unavailable, what safe default is required?

The same questions apply to graph traversal, agent tools, compliance workflows, orders, and security checks. This is the practical value of motif-level representation: it gives the system a reusable grammar for closure.

8 Limitations

The experiments are synthetic and deterministic. They are designed to isolate closure semantics rather than measure natural-language extraction performance on a broad benchmark. The baselines are intentionally simple, because the claim under test is structural: accepting local success as terminal validity is unsafe. Future work should compare against LLM agents, workflow orchestrators, security policy engines, and graph-reasoning systems on larger natural traces.

The current rule also treats mandatory violated obligations as dominant over unknown obligations. In real deployments, evidence may conflict rather than merely disappear. A natural DLR-O-3 extension would add conflict obligations and ledger revision. Examples include one shard asserting an edge while another reports deletion, one sanctions provider clearing a vendor while another flags it, or a payment processor reporting capture while the ledger remains unsettled.

9 Conclusion

DLR-O completes the reliability endpoint of the DLR stack. DLR represents process structure. DLR-lambda recovers that structure from distributed evidence. DLR-O decides whether a terminal claim is licensed by the mandatory obligations generated by the process.

The completed terminal logic is:

$$\begin{aligned} \text{satisfied mandatory obligations} &\rightarrow \textsc{valid} \\ \text{violated mandatory blocking obligation} &\rightarrow \textsc{invalid} \\ \text{unknown mandatory blocking obligation} &\rightarrow \textsc{unknown} \end{aligned}$$

This is the reliability primitive: a system is not reliable because it always answers. It is reliable because it knows when it is not allowed to close.

A Reproduction Commands

The experiments can be regenerated with:

```
npm run run:dlr-o
npm run run:dlr-o-stress
```

The generated reports are:

- `dlr_obligation_transfer.md`
- `artifacts/dlr_obligation_transfer.json`
- `dlr_obligation_stress.md`
- `artifacts/dlr_obligation_stress.json`

References

[1] Nikit Phadke. *Recursive Log Diagnosis with ProcessIR: A DLR-Lambda Experiment on Loghub.* 2026.

[2] Nikit Phadke. *From Batch Reduction to Investigative Fixpoint: A DLR-Lambda-2 Experiment on Marker-Free Log Diagnosis.* 2026.

[3] Nikit Phadke. *DLR-Lambda-3 Motif Obligation Calculus.* 2026.

[4] Nikit Phadke. *Trace-to-Causal-IR Diagnosis and Hidden Preconditions.* 2026.